



Versioning Data, Models, and Parameters

A comprehensive guide to tracking ML experiments for reproducibility and productionization

Why Versioning Matters

ML experiments constantly evolve across multiple dimensions:

Experiment Complexity

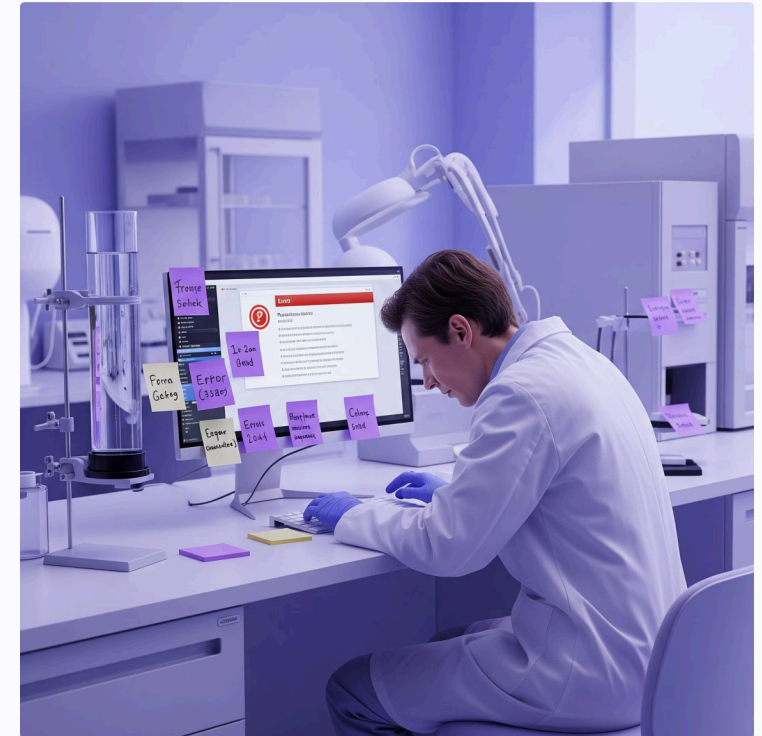
ML workflows involve constantly changing **datasets, code, and hyperparameters**

Team Collaboration

Versioning creates a **single source of truth** that enables multiple engineers to work together

Regulatory Requirements

Industries like finance and healthcare **legally require** complete lineage tracking



Without versioning, reproducing results becomes impossible

The Three Pillars of ML Versioning

A complete versioning strategy must address all three components:

1

Data

Raw and processed datasets, including transformations and feature engineering steps

2

Models

Trained weights, architectures, frameworks, and inference code

3

Parameters

Configurations, hyperparameters, random seeds, and environment settings

⊗ **Missing even one pillar breaks reproducibility!** An experiment with tracked models and code but missing dataset versions cannot be replicated accurately.

Versioning Data

Track with Unique Identifiers

Use dataset IDs or content-based hashes (SHA-256) to uniquely identify each version

Version Control Solutions

- **DVC (Data Version Control)** - Git extension for datasets
- Git LFS - for smaller feature stores
- LakeFS - Git-like versioning for cloud storage

Log Transformations

Always document preprocessing steps and feature engineering transformations



Proper data versioning prevents the "mystery dataset" problem

Versioning Models

Save Complete Artifacts

Ensure both model weights AND architecture configuration are preserved together

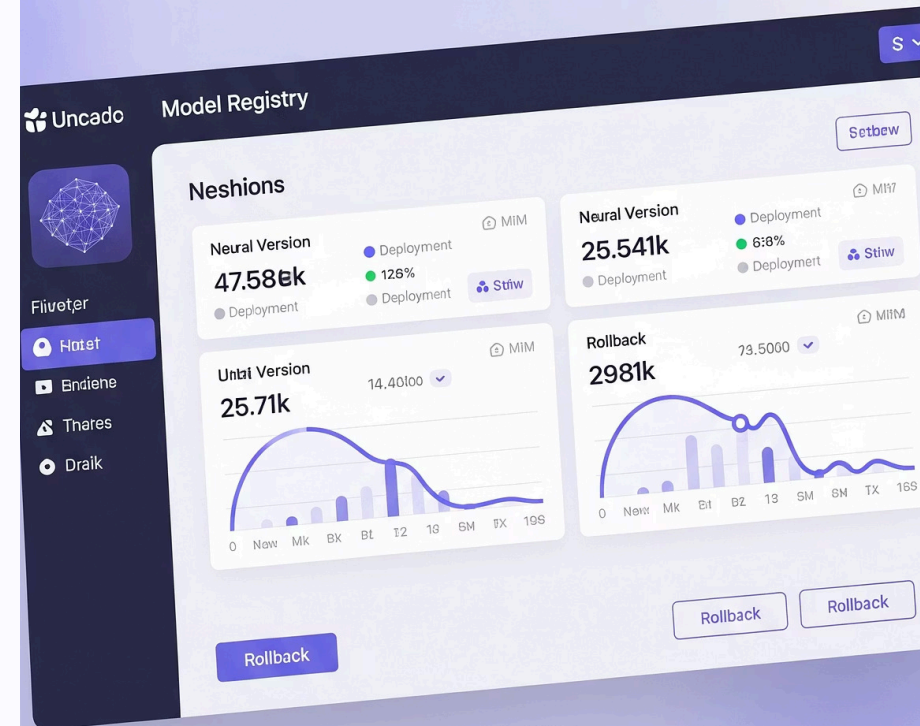
Implement Registry System

[MLflow Model Registry](#) provides version control and staging workflows (Dev → Staging → Production)

Enable Model Lifecycle

Support promotion, rollback, and audit capabilities for each version

A robust model versioning system allows for quick rollbacks when new versions fail in production, saving critical business functions.



Versioning Parameters

Parameters often explain **why** a model performed differently despite using the same architecture and data.

Track Hyperparameters

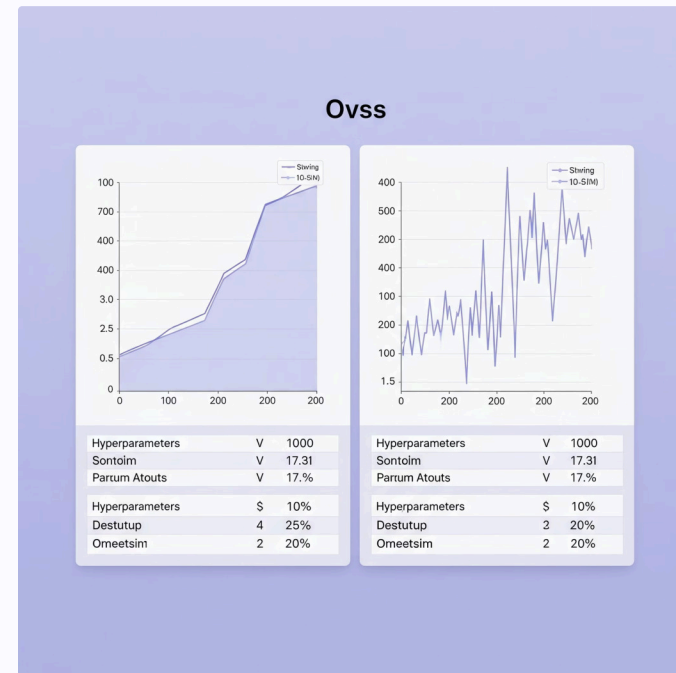
- Learning rates
- Batch sizes
- Loss functions
- Optimizer configurations

Use Config Files

Store in JSON/YAML files that can be version controlled with Git

Log Random Seeds

Essential for true reproducibility in stochastic processes



[MLflow](#) and other experiment trackers provide dedicated methods for parameter logging: `log_param()` and `log_params()`

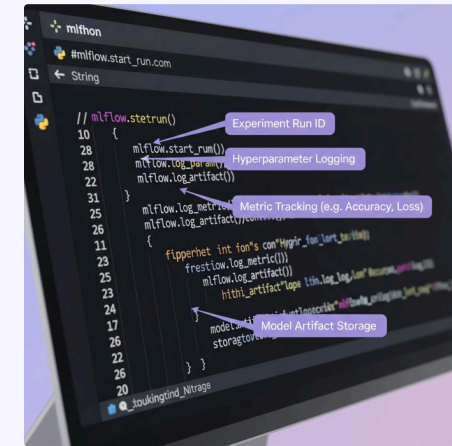
Example: Comprehensive Versioning with MLflow

```
# Log parameters individually
mlflow.log_param("learning_rate", 0.001)
mlflow.log_param("batch_size", 128)

# Log configuration files
mlflow.log_artifact("params.yaml")

# Log dataset version information
mlflow.log_artifact("dataset_hash.txt")

# Log the model itself
mlflow.pytorch.log_model(model, "model")
```



This example captures **all three pillars** of versioning in a single run:

- Parameters through `log_param` calls
- Dataset identification via hashed reference
- Model architecture and weights with `log_model`

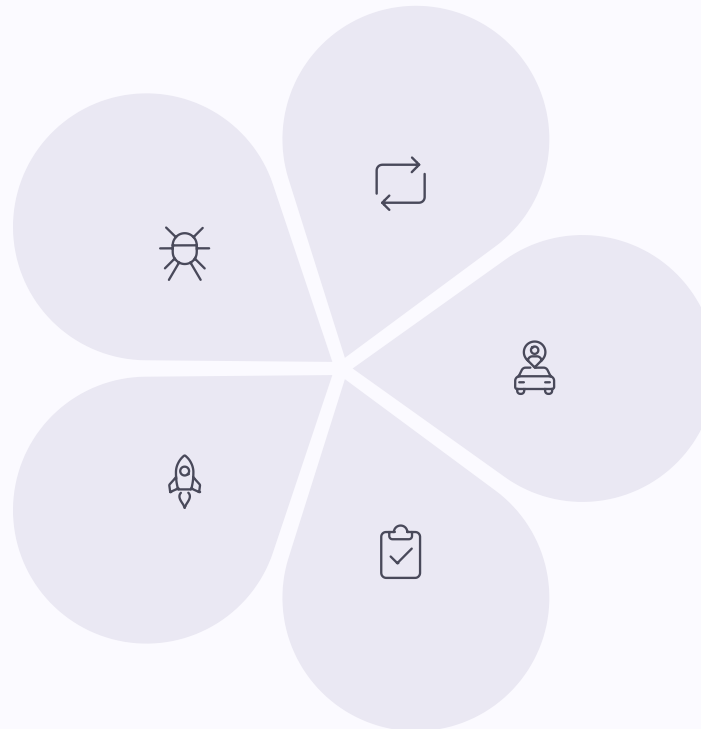
Benefits of Comprehensive Versioning

Debugging

Quickly understand why performance improved or regressed between versions

CI/CD for ML

Build automated pipelines with clear inputs and outputs



Reproducibility

Recreate past experiments with exact fidelity, even years later

Collaboration

Enable teamwork across data scientists, ML engineers and platform teams

Compliance

Generate audit trails and documentation for regulatory requirements

Organizations with mature versioning capabilities deploy ML systems **2-3x faster** than those without structured versioning practices.

Common Tools for Versioning

MLflow Registry

Centralizes model storage, lineage tracking and deployment staging

DVC & Git LFS

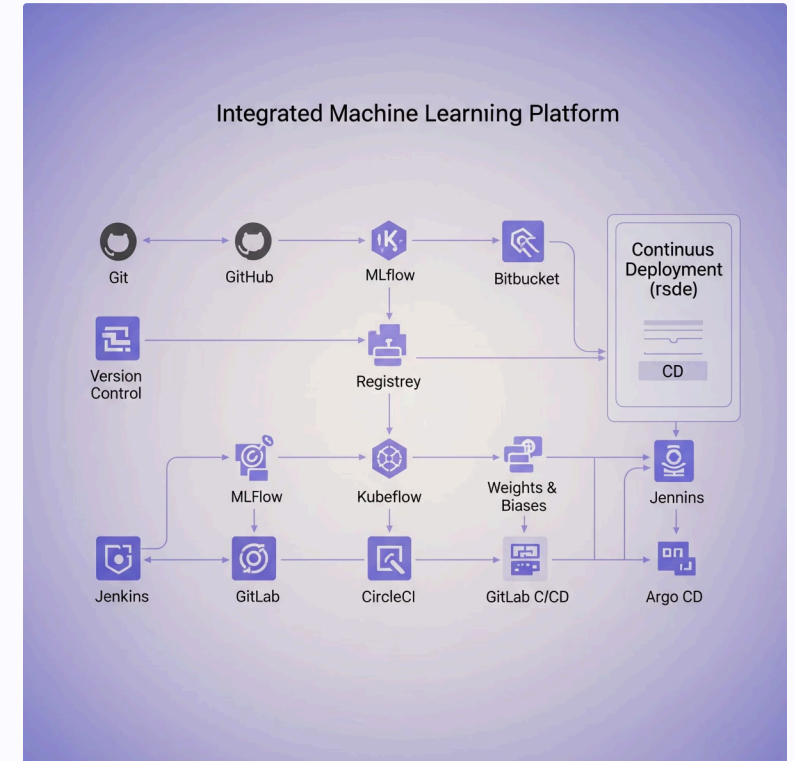
Provides Git-like versioning for datasets and large files

Weights & Biases

Rich experiment tracking with parameter logging and visualization

LakeFS

Git semantics for data lakes and object storage (S3, GCS)



Most organizations use a [combination of tools](#), integrating them with Git for complete lineage tracking across the ML lifecycle.



Challenges in Versioning



Storage Costs

Large datasets lead to storage bloat and duplication concerns



Syncing Components

Maintaining relationships between data, code, and model versions



Environment Consistency

Ensuring reproducibility across different compute environments



Governance vs. Flexibility

Balancing strict controls with researcher productivity

Organizations must carefully design their versioning strategy to address these challenges while supporting both experimentation and production needs.

Key Takeaways

1 Versioning is Non-Negotiable

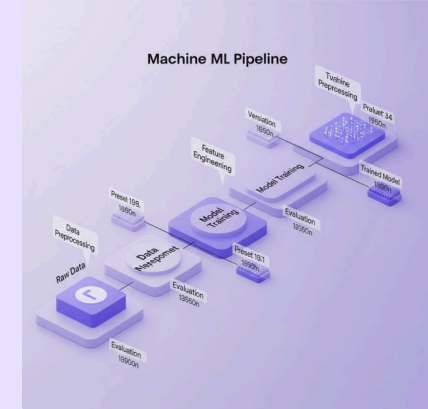
Reproducible ML requires systematic tracking of experiments

2 Three Pillars Approach

Track **data, models, and parameters**
together for complete reproducibility

3 Leverage Modern Tools

MLflow, DVC, W&B, and other platforms simplify versioning workflows



Without proper versioning, AI infrastructure becomes fundamentally **unreliable**. Invest in versioning early to enable scaling, collaboration, and compliance.

Next steps: Audit your current versioning practices and identify gaps across the three pillars.